



AROR UNIVERSITY
OF ART, ARCHITECTURE,
DESIGN & HERITAGE,
SUKKUR, SINDH

Software Processes

Prof. Dr. Faheem Akhtar Rajput





Objectives

- Understand the concepts of software processes and software process models;
- Have been introduced to three general software process models and when they might be used;
- Know about the fundamental process activities of software requirements engineering, software development, testing, and evolution;
- Understand why processes should be organized to cope with changes in the software requirements and design;
- Understand the notion of software process improvement and the factors that affect software process quality.



The software process

- A structured set of activities required to develop a software system.
 - **Software Specification:** The functionality of the software and constraints on its operation must be defined;
 - **Software Design and Implementation:** The software to meet the specification must be produced.;
 - **Software Validation:** The software must be validated to ensure that it does what the customer wants;
 - **Software Evolution:** The software must evolve to meet changing customer needs.
- A software process model is an abstract representation of a process. It presents a description of a process from some particular perspective.

Software process descriptions

- When we describe and discuss processes, we usually talk about the activities in these processes such as specifying a data model, designing a user interface, etc. and the ordering of these activities.
- However, when describing processes, it is also important to describe who is involved, what is produced, and conditions that influence the sequence of activities.

Software process descriptions

- Process descriptions may also include:
 - **Products or deliverables** are the outcomes of a process activity.
 - For example, the outcome of the activity of architectural design may be a model of the software architecture.
 - **Roles** reflect the responsibilities of the people involved in the process.
 - Examples of roles are project manager, configuration manager, and programmer.
 - **Pre- and postconditions** are conditions that must hold before and after a process activity has been enacted or a product produced.
 - For example, before architectural design begins, a precondition may be that the consumer has approved all requirements; after this activity is finished, a postcondition might be that the UML models describing the architecture have been reviewed.

Plan-driven and Agile Process

- Some times software process are categorized as either plan-driven or agile process.
- **Plan-driven** processes are processes where all the process activities are planned in advance and process is measured against plan.
- In **agile processes**, planning is incremental and it is easier to change the process to reflect changing customer requirements.
- In practice, most practical processes include elements of both plan-driven and agile approaches.
- There are no right or wrong software processes.
- As Boehm and Turner (2003) discuss, each approach is suitable for different type of software.
- Generally, you need to find a balance between plan-driven and agile processes.

A decorative header consisting of a row of seven hexagons in various shades of blue, white, and orange, with some hexagons having orange outlines. Below this, on the right side, are two more hexagons: a dark blue one with an orange outline and a solid orange one.

The software process models

- The software process model is an abstract representation of a software process.
- Each process model represents a process from a particular perspective, and thus provides only partial information about that process.
- The Generic models are not definitive descriptions of software processes. Rather, they are abstractions of the process that can be used to explain different approaches to software development.

Generic software process models

- In Generic software models we will focus on the following models:
- **The waterfall model:** Plan-driven model. Separate and distinct phases of specification and development.
 - This takes the fundamental process activities of specification, development, validation and evolution and presents them as separate process phase such as requirements specification, software design, implementation, testing and so on.
- **Incremental development:** Specification, development and validation are interleaved. May be plan-driven or agile.
 - This approach interleaves the activities of specification, development and validation. The system is developed as a series of versions (increments), with each version adding functionality to the previous version.
- **Reuse-oriented software engineering:** The system is assembled from existing components. May be plan-driven or agile.
 - This approach is based on the existence of a significant number of reusable components. The system development process focuses on integrating these components into a system rather than developing them from scratch.

Generic software process models (Old Version)

- In Generic software models we will focus on the following models:
- **The waterfall model:** Separate and distinct phases of specification and development.
 - This takes the fundamental process activities of specification, development, validation and evolution and presents them as separate process phase such as requirements specification, software design, implementation, testing and so on.
- **Evolutionary development:** Specification, development and validation are interleaved.
 - This approach interleaves the activities of specification, development and validation. An initial system is rapidly developed from abstract specifications. This is then refined with customer input to produce a system that satisfies the customer's needs.
- **Component-based software engineering:** The system is assembled from existing components.
 - This approach is based on the existence of a significant number of reusable components. The system development process focuses on integrating these components into a system rather than developing them from scratch.



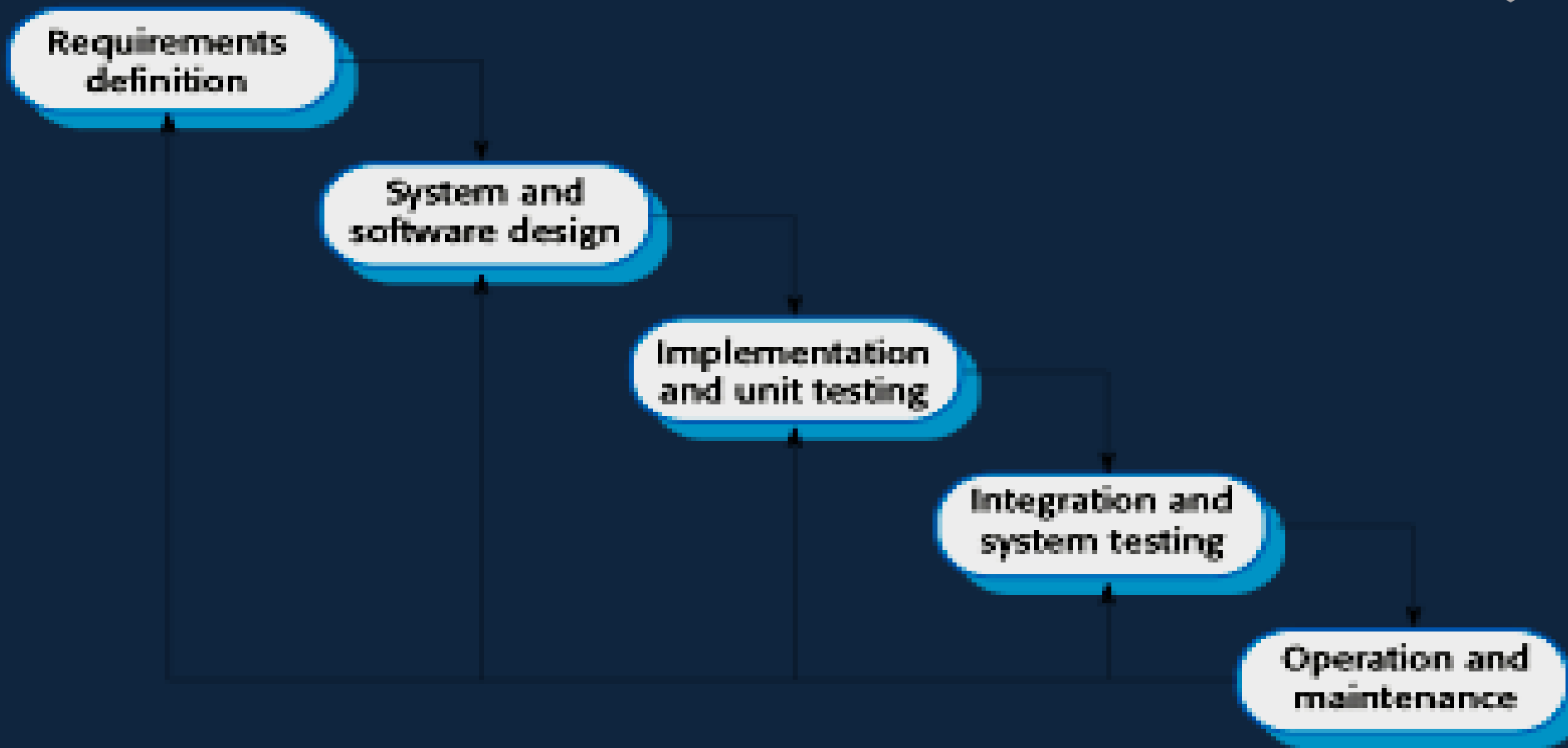
Software process models

- **Waterfall model**
 - Plan-driven model. Separate and distinct phases of specification and development.
- **Incremental development**
 - Specification, development and validation are interleaved. May be plan- driven or agile.
- **Integration and configuration**
 - The system is assembled from existing configurable components. May be plan-driven or agile.
- In practice, most large systems are developed using a process that incorporates elements from all of these models.

Generic Software process models (Cont.)

- In practice, most large systems are developed using a process that incorporates elements from all of these models.
- These three generic models are widely used in current software engineering practice.
- They are not mutually exclusive and are often used together, especially for large system development.
- There are many variants of these models e.g. formal development where a waterfall-like process is used but the specification is a formal specification that is refined through several stages to an implementable design.

Waterfall model



Waterfall model phases

- The First published model of the software development process was derived from more general system engineering processes (Royce, 1970).
- Also called the *Classic Life Cycle* or *Linear Sequential Model*
- The principal stages of the model map onto fundamental development activities:
 - *Requirements analysis and definition:*
 - The system's services, constraints and goals are established by consultation with system users.
 - They are then defined in detail and serve as a system specifications.
 - *System and software design:*
 - The system design process partitions the requirements to either hardware or software requirements.
 - It establishes an overall system architecture.
 - Software design involves identifying and describing the fundamental software abstractions and their relationships.

Waterfall model phases (Cont.)

- ***Implementation and unit testing:***

- During this stage, the software design realised as a set of programs or program units.
- Unit testing involves verifying that each unit meets its specification.

- ***Integration and system testing:***

- The individual program units or programs are integrated and tested as a complete system to ensure that the software requirements have been met.
- After testing, the software system is delivered to the customer.

- ***Operation and maintenance:***

- Normally (although not necessarily) this is the longest life-cycle phase.
- The system is installed and put into practical use.
- Maintenance involves correcting errors which were not discovered in earlier stages of the life cycle, improving the implementation of system units and enhancing the system's services as new requirements are discovered.

Waterfall model phases (Cont.)

- In Principle, the result of each phase is one or more documents that are approved ('Signed off').
- The following phase should not start until the previous phase has finished.
- In Practice, these stages overlap and feed information to each other.
- During design, problems with requirements are identified; during coding design problems are found and so on.
- The software process is not a simple linear model but involves a sequence of iterations of the development activities.
- The main drawback of the waterfall model is the difficulty of accommodating change after the process is underway.
 - One phase has to be complete before moving onto the next phase.

Applicability of Waterfall model

- The waterfall model is only appropriate for some types of system:
 - **Embedded systems** where the software has to interface with hardware systems.
 - Because of the inflexibility of hardware, it is not usually possible to delay decisions on the software's functionality until it is being implemented.
 - **Critical systems** where there is a need for extensive safety and security analysis of the software specification and design.
 - In these systems, the specification and design documents must be complete so that this analysis is possible.
 - Safety related problems in the specification and design are usually very expensive to correct at the implementation stage.

Applicability of Waterfall model

- The waterfall model is only appropriate for some types of system:
 - **Large software systems** that are part of broader engineering systems developed by several partner companies.
 - The hardware in the systems may be developed using a similar model, and companies find it easier to use a common model for hardware and software.
 - Furthermore, where several companies are involved, complete specifications may be needed to allow for the independent development of different subsystems.
- The waterfall model is not the right process model in situations where informal team communication is possible and software requirements change quickly.

Advantages of Waterfall Model

- The advantages of the waterfall model are that documentation is produced at each phase and that it fits with other engineering process models.
- The waterfall model should only be used when the requirements are well understood and unlikely to change radically during system development.
- However, the waterfall model reflects the type of process model used in other engineering projects.
- Consequently, software processes based on this approach are still used for software development, particularly when the software projects is part of the larger system engineering project.

Waterfall model problems

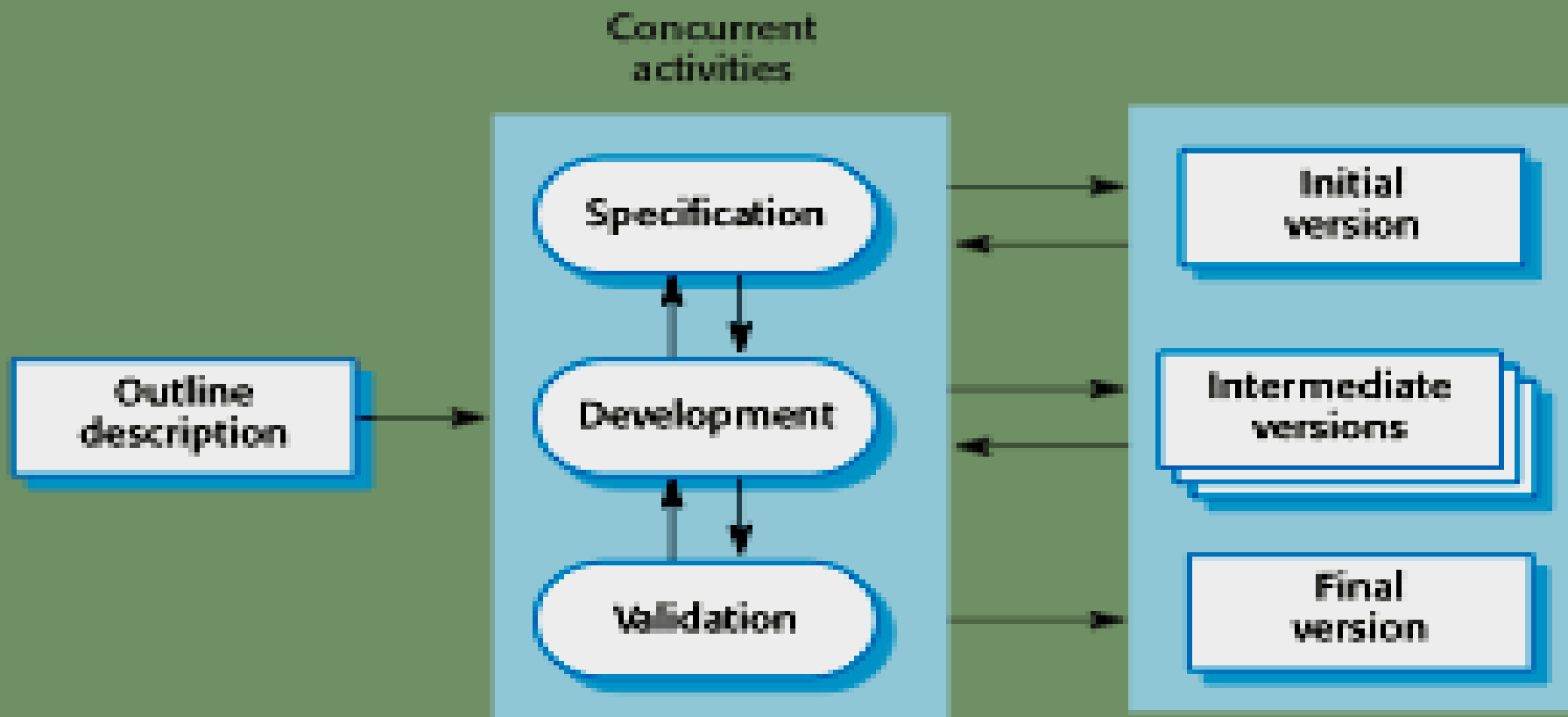
- Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.
 - Therefore, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process.
 - Few business systems have stable requirements.
- The waterfall model is mostly used for large systems engineering projects where a system is developed at several sites.
 - In those circumstances, the plan-driven nature of the waterfall model helps coordinate the work.



Incremental development

- Incremental development is based on the idea of developing an initial implementation, exposing this to user comment and refining it through many versions until an adequate system has been developed.
- Specification, development and validation are interleaved rather than separate, with rapid feedback across activates.

Incremental development



Incremental development (Cont.)

- Incremental development in some form is now the most common approach for the development of application systems and software products.
- This approach can be either plan-driven, agile or, more usually, a mixture of these approaches.
- In a plan-driven approach, the system increments are identified in advance; if an agile approach is adopted, the early increments are identified, but the development of later increments depends on progress and customer priorities.



Incremental development (Cont.)

- There are two fundamental types of incremental development:
- **Exploratory development**
 - Objective is to work with customers and to evolve a final system from an initial outline specification. Should start with well-understood requirements and add new features as proposed by the customer.
- **Throw-away prototyping**
 - Objective is to understand the system requirements. Should start with poorly understood requirements to clarify what is really needed.

Incremental development (Cont.)

- An Incremental approach to software development is often more effective than waterfall approach in producing systems that meet the immediate needs of customers.
- The advantage of a software process that is based on an incremental approach is that the specification can be developed incrementally.
- As users develop a better understanding of their problem, this can be reflected in the software system.

Incremental development benefits

- The cost of accommodating changing customer requirements is reduced.
 - The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
- It is easier to get customer feedback on the development work that has been done.
 - Customers can comment on demonstrations of the software and see how much has been implemented.
- More rapid delivery and deployment of useful software to the customer is possible.
 - Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

Incremental development (Cont.)

- However, from an engineering and management perspective, the incremental approach has two problems.
 - *The process is not visible*: Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.
 - *Systems are often poorly structured*: Continual change tends to corrupt the software structure. Incorporating software changes becomes increasingly difficult and costly.

To reduce structural degradation and general code messiness, agile methods suggest that you should regularly refactor (improve and restructure) the software.



Incremental development (Cont.)

- Problems
 - Lack of process visibility;
 - Systems are often poorly structured;
 - Special skills (e.g. in languages for rapid prototyping) may be required.
- Applicability
 - For small or medium-size interactive systems;
 - For parts of large systems (e.g. the user interface);
 - For short-lifetime systems.



Integration and configuration

- Based on software reuse where systems are integrated from existing components or application systems (sometimes called COTS -Commercial-off-the-shelf) systems).
- Reused elements may be configured to adapt their behaviour and functionality to a user's requirements
- Reuse is now the standard approach for building many types of business system
 - Reuse covered in more depth in Chapter 16.



Reuse-oriented software engineering

- In the majority of software projects, there is some software reuse.
- This usually happens informally when people working on the project know of designs or code which is similar to that required.
- They look for these, modify them as needed and incorporate them into their systems.
- In the incremental approach, reuse is often essential for rapid system development.
- This informal reuse takes place irrespective of the development process that is used.

Reuse-oriented software engineering (Cont.)

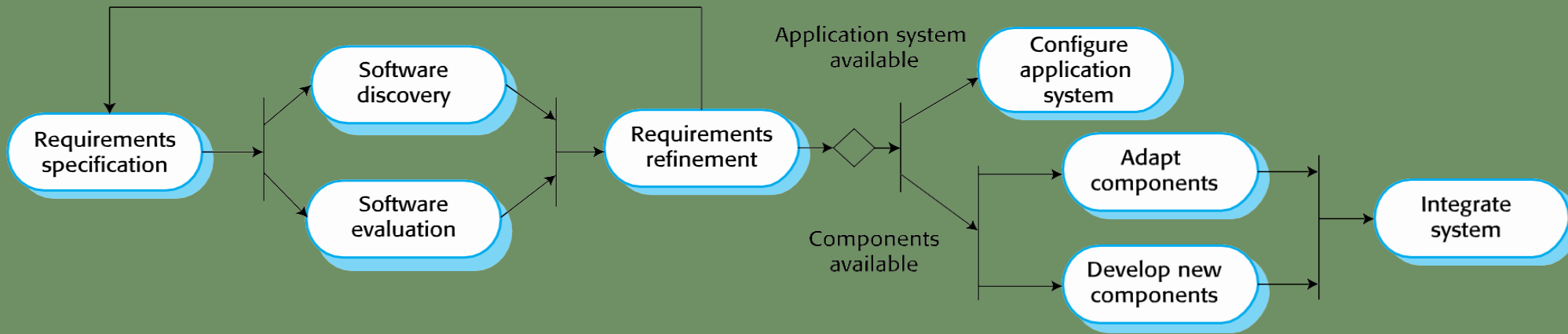
- However, in the last few years, an approach to software development called *Reuse-oriented software engineering* or *component-based software engineering* (CBSE), which relies on reuse, has emerged and it is becoming increasingly used.
- This reuse-oriented approach relies on a large base of reusable software components and some integrating framework for these components.
- Sometimes, these components are systems in their own right (COTS or commercial off-the-shelf systems) that may provide specific functionality such as text formatting or numeric calculations.



Types of reusable software components

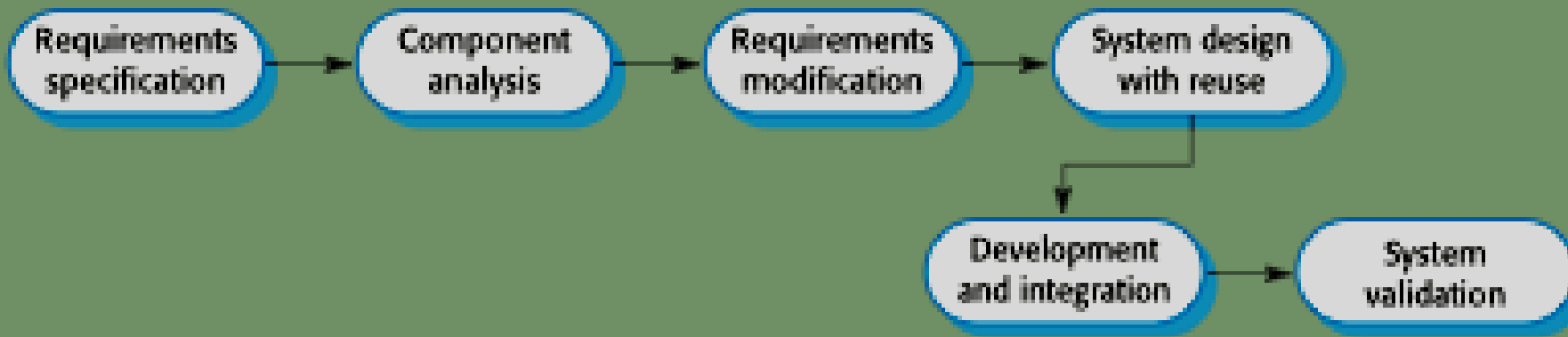
- Stand-alone application systems that are configured for use in a particular environment. These systems are general-purpose systems that have many features, but they have to be adapted for use in a specific application.
- Collections of objects that are developed as a component or as a package to be integrated with a component framework such as the Java Spring framework.
- Web services that are developed according to service standards and that are available for remote invocation over the Internet.

Reuse-oriented software engineering



Reuse-oriented development

The generic process model for CBSE



Reuse-oriented software engineering (Cont.)

- These stages are:
 - *Requirement Specification:*
 - The initial requirements for the system are proposed.
 - These do not have to be elaborated in detail but should include brief descriptions of essential requirements and describe system features.
 - *Software discovery and evaluation:*
 - Given an outline of the software requirements, a search is made for components and systems that provide the functionality required.
 - Candidate components and systems are evaluated to see if they meet the essential requirements and if they are generally suitable for use in the system.

Reuse-oriented software engineering (Cont.)

- *Requirement refinement:*

- During this stage, the requirements are refined using information about the reusable components and applications that have been discovered.
- The requirements are modified to reflect the available components, and the system specifications is re-defined.
- Where modifications are impossible, the component analysis is activity may be re-entered to search for alternative solutions.

- *Application system configuration:*

- If an off-the-shelf system that meets the requirements is available, it may then be configured for use to create the new system

- *Component adaption and integrations:*

- If there is no Off-the-shelf system, individual reusable components may be modified and the new component developed.
- These are then integrated to create the system.



Advantages and disadvantages

- Reduced costs and risks as less software is developed from scratch
- Faster delivery and deployment of system
- But requirements compromises are inevitable so system may not meet real needs of users
- Loss of control over evolution of reused system elements

Component-based software engineering

- Component-based software engineering has the obvious advantage of reducing the amount of software to be developed and so reducing the cost and risk.
- It usually also leads to faster delivery of the software.
- However, requirements compromises are inevitable and this may lead to a system that does not meet the real needs of users.
- Furthermore, some control over the system evolution is lost as new versions of reusable components are not under the control of the organization using them.

Component-based software engineering (Cont.)

- CBSE has much in common with an emerging approaches to system development that is based on integerting web services from a range of suppliers.
- This approach is becoming increasingly used as component standards have emerged.



Process activities

- Real software processes are inter-leaved sequences of technical, collaborative and managerial activities with the overall goal of specifying, designing, implementing and testing a software system.
- The four basic process activities of specification, development, validation and evolution are organized differently in different development processes. In the waterfall model, they are organized in sequence, whereas in incremental development they are inter-leaved.



Process activities (Cont.)

- The four basic process activities are:
 - Software specification
 - Software design and implementation
 - Software validation
 - Software evolution
- In the waterfall model, they are organized in a sequence, whereas in incremental development they are interleaved.
- How these activities are carried out depends on the type of software, people, and organizational structure involved.
- There is no right or wrong way to organize these activities and my goal in this section is simply to provide you with an introduction to how they can be organized.



Software specification

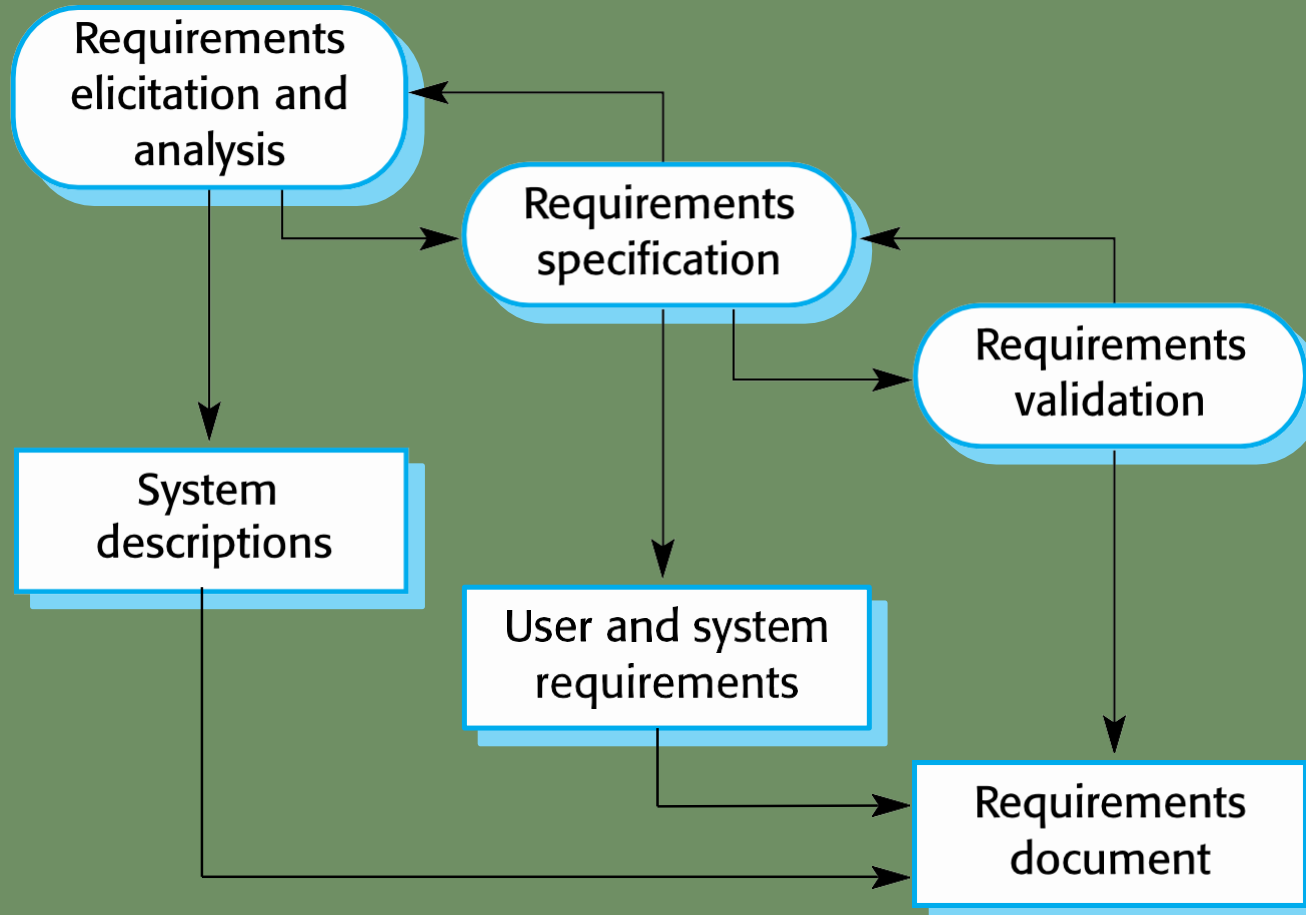
- Software Specification or requirements engineering is the process of understanding and defining what services are required from the system and identifying the constraints on the system's operation and development.
- Requirement engineering is a particularly critical stage of the software process, as mistakes made at this stage inevitably lead to later problems in the system design and implementations.



Software specification

- The process of establishing what services are required and the constraints on the system's operation and development.
- The requirement engineering process leads to the production of requirement document that is the specification for the system.
- Requirements are usually presented at two levels of detail in this document.
- End-user and customers need a high-level statement of the requirements; system developers need a more detailed system specification.

The requirements engineering process





Software specification

- The process of establishing what services are required and the constraints on the system's operation and development.
- **Requirements engineering process**
 - **Feasibility study**
 - Is it technically and financially feasible to build the system?
 - **Requirements elicitation and analysis**
 - What do the system stakeholders require or expect from the system?
 - **Requirements specification**
 - Defining the requirements in detail
 - **Requirements validation**
 - Checking the validity of the requirements

Software specification (Cont.)

- The three main phases in the requirements engineering process are:
 - **Requirements elicitation and analysis**
 - This is the process of deriving the system requirements through observation of existing systems, discussions with potential users and procurers, take analysis and so on.
 - This may involve the development of one or more system models and prototypes.
 - These help the analyst understand the system to be specified.

Software specification (Cont.)

- Requirement Specification

- The activity of translating the information gathered during the analysis activity into a document that defines a set of requirements.
- Two type of requirements may be included in this document.
 - *User requirements* are abstract statements of the system requirements for the customer and end-user of the system;
 - *System requirements* are a more detailed description of the functionality to be provided.



Software specification (Cont.)

■ Requirements validation

- This activity checks the requirements for realism, consistency and completeness.
- During this process, errors in the requirements document are inevitably discovered.
- It must then be modified to correct these problems.

Software specification (Cont.)

- Of course, the activities in the requirement process are not simply carried out in a strict sequence.
- Requirement analysis continues during definition and specification, and new requirements come to light throughout the process.
- Therefore, the activities of analysis, definition and specification are interleaved.
- In agile methods, such as extreme programming, requirements are developed incrementally according to user priorities and the elicitation of requirements comes from users who are part of the development team.
- Requirements are informally specified for each increment of the system just before that increment is developed.

Software specification (Cont.)

■ Extreme programming (XP)

- XP is “a light-weight methodology for small to medium-sized teams developing software in the face of vague or rapidly changing requirements.”
- Alternative to “heavy-weight” software development models
(which tend to avoid change and customers)
- "Extreme Programming turns the conventional software process sideways.
- Rather than planning, analyzing, and designing for the far-flung future, XP programmers do all of these activities a little at a time throughout development.”—IEEE Computer , October 1999



Software design and implementation

- The process of converting the system specification into an executable system.
- Software design
 - Design a software structure that realises the specification;
- Implementation
 - Translate this structure into an executable program;
- The activities of design and implementation are closely related and may be inter-leaved.

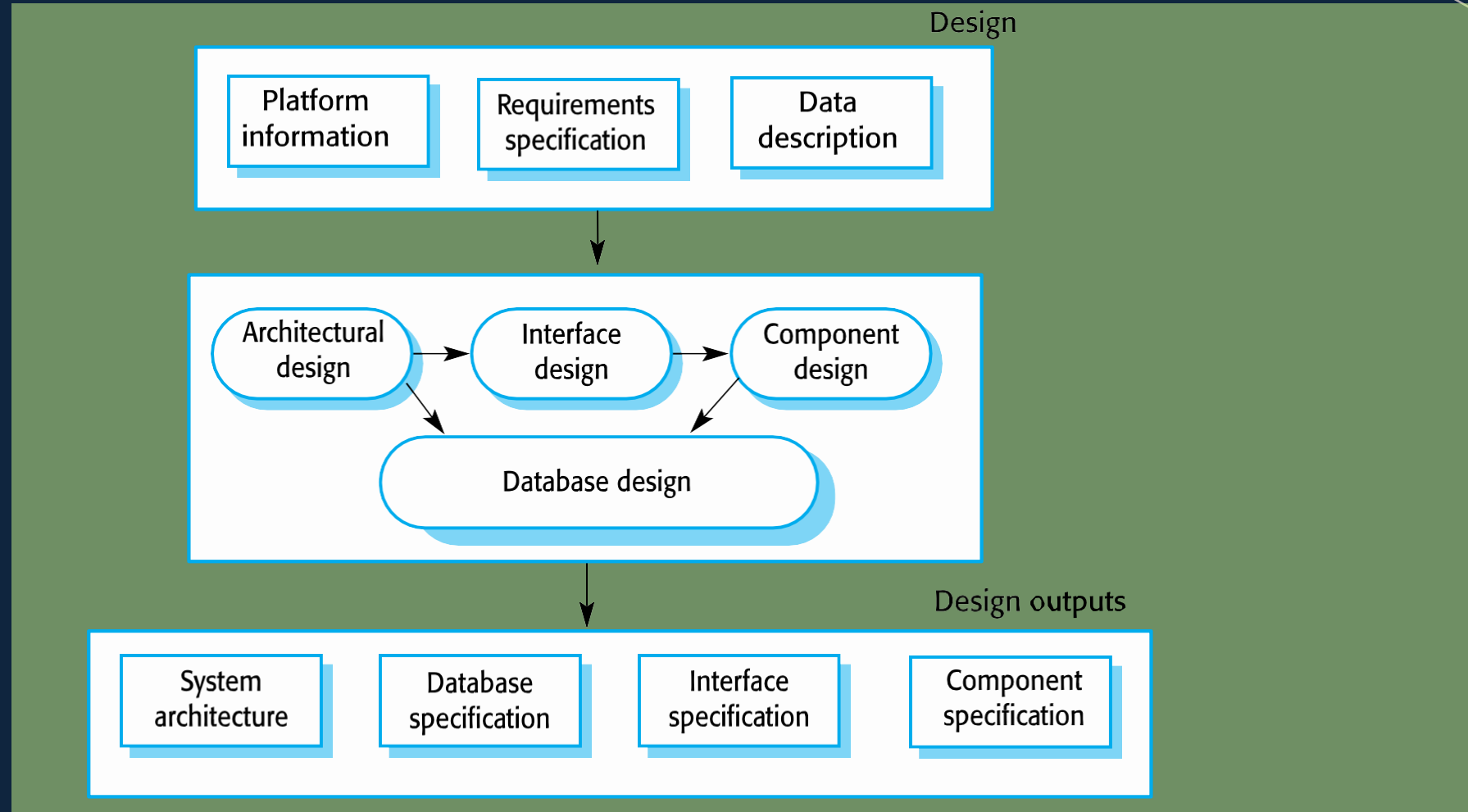
Software design and implementation (Cont.)

- The implementation stage of software development is the process of converting the system specification into an executable system.
- Sometimes, this involves separate activities of software design and programming.
- However, if an agile approach to development is used, design and implementation are interleaved, with no formal design document produced during the process.
- Of course, the software is still designed, but the design is recorded informally on whiteboards and programmer's notebooks.

Software design and implementation (Cont.)

- A software design is the description of the software to be implemented, the data model and structure used by the system, the interface between system components and, sometimes, the algorithm used.
- Designers do not arrive at a finished design immediately but develop the design iteratively.
- They add formality and detail as they develop their design with constant backtracking to correct earlier design.

The software design process



Software design and implementation (Cont.)

- An abstract model of this process shows the inputs to the design process, process activities, and the documents produced as outputs from this process.
- The design process activities are both interleaved and interdependent.
- New information about the design is constantly being generated, and this affects previous design decisions.
- Design rework is therefore inevitable.

Software design and implementation (Cont.)

- Most software interfaces with other software systems. These include the operating system, database, middleware, and other application systems.
- These make up the ‘software platform’, the environment in which the software will execute.
- Information about this platform is an essential input to the design process, as designers must decide how best to integrate it with the software environment.
- If the system is to process existing data, then the description of that data may be included in the platform specification.
- Otherwise, the data description must be an input to the design process so that the system data organization to be defined.



Design process activities

- The activities in the design process vary, depending on the type of the system being developed.
 - For example, real-time systems require timing design but may not include a database so there is no database design involved.
- There are four activities that may be part of the design process of information systems:
 - **Architectural design:** where you identify the overall structure of the system, the principal components (sometimes called sub-systems or modules), their relationships and how they are distributed.
 - **Database design:** where you design the system data structures and how these are to be represented in a database.
 - Again, the work here depends on whether an existing database is to be reused or a new database is to be created.

Design process activities (Cont.)

- There are four activities that may be part of the design process of information systems:
 - **Interface design:** where you define the interfaces between system components. This interface specifications must be unambiguous.
 - With a precise interface, a component may be used by other components without them having to know how it is implemented.
 - Once interface specifications are agreed, the components can be separately designed and developed.
 - **Component selection and design:** where you search for reusable components and, if no suitable components are available, design new software components.
 - The design at this stage, may be a simple component description with the implementation details left to the programmer.
 - Alternatively, it may be a list of changes to be made to a reusable component or detailed design model expressed in UML.
 - The design model may then be used to automatically generate an implementation.

Design process activities (Cont.)

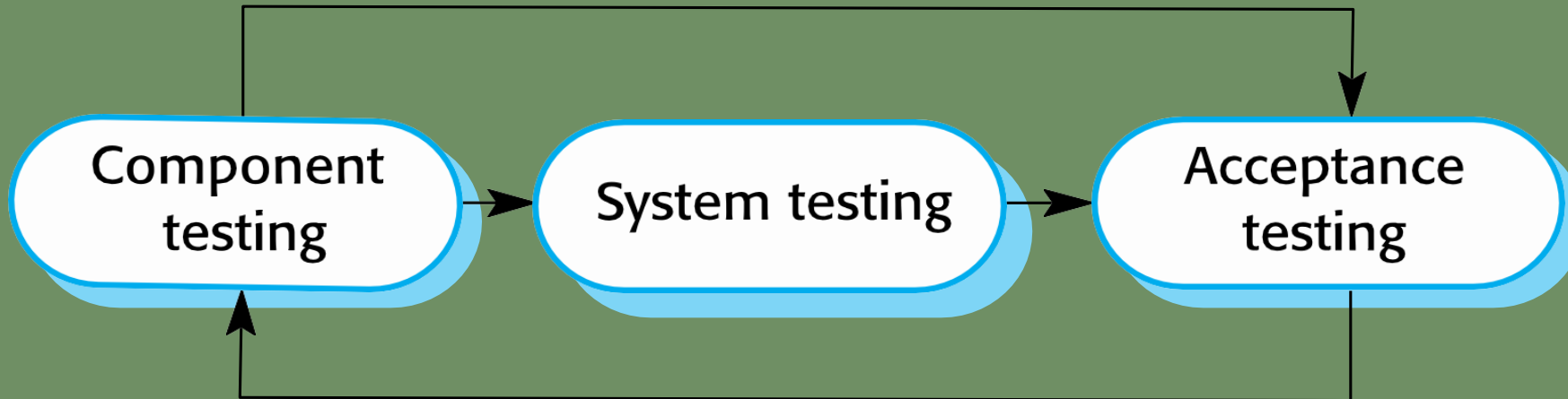
- These activities lead to the design outputs, which are also depicted in the general model of the design process.
- For critical systems, the outputs of the design process are detailed design documents setting out precise and accurate descriptions of the system.
- If a *model-driven* approach (Chapter 5) is used, the design outputs are the design diagrams.
- Where *agile methods* of development are used, the output of the design process may not be separate specification documents but may be represented in the code of the program.



Software validation

- Verification and validation (V & V) is intended to show that a system conforms to its specification and meets the requirements of the system customer.
- Involves checking and review processes and system testing.
- System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system.

The testing process





Testing stages

- Component testing
 - The components making up the system are tested by the people developing the system.
 - Each component is tested independently, without other system components.
 - Components may be simple entities such as functions or object classes or may be coherent groupings of these entities.
 - Test automation tools, such as JUnit for Java, that can rerun tests when new versions of the component are created, are commonly used

Testing stages

- System testing
 - System components are integrated to create a complete system.
 - This process is concerned with finding errors that result from unanticipated interactions between components and component interface problems.
 - It is also concerned with showing that the system meets its functional and non-functional requirements, and testing the emergent system properties.
 - For large systems, this may be a multistage process where components are integrated to form subsystems that are individually tested before these subsystems are integrated to form the final system.

Testing stages

- Customer testing

- This is the final stage in the testing process before the system is accepted for operational use.
- The system is tested by the system customer (or potential customer) rather than with simulated test data.
 - For custom-built software, customer testing may reveal errors and omissions in the system requirements definition, because the real data exercise the system in different ways from the test data.

Customer testing may also reveal requirements problems where the system's facilities do not really meet the users' needs or the system performance is unacceptable.
 - For products, customer testing shows how well the software product meets the customer's needs.

Testing stages (Cont.)

- Ideally, component defects are discovered early in the testing process, and interface problems are found when the system is integrated.
- However, as defects are discovered, the program must be debugged, and this may require other stages in the testing to be repeated.
- Errors in program components, say, may come to light during system testing.
- The process is therefore an iterative one with information being fed back from later stages to earlier parts of the process.



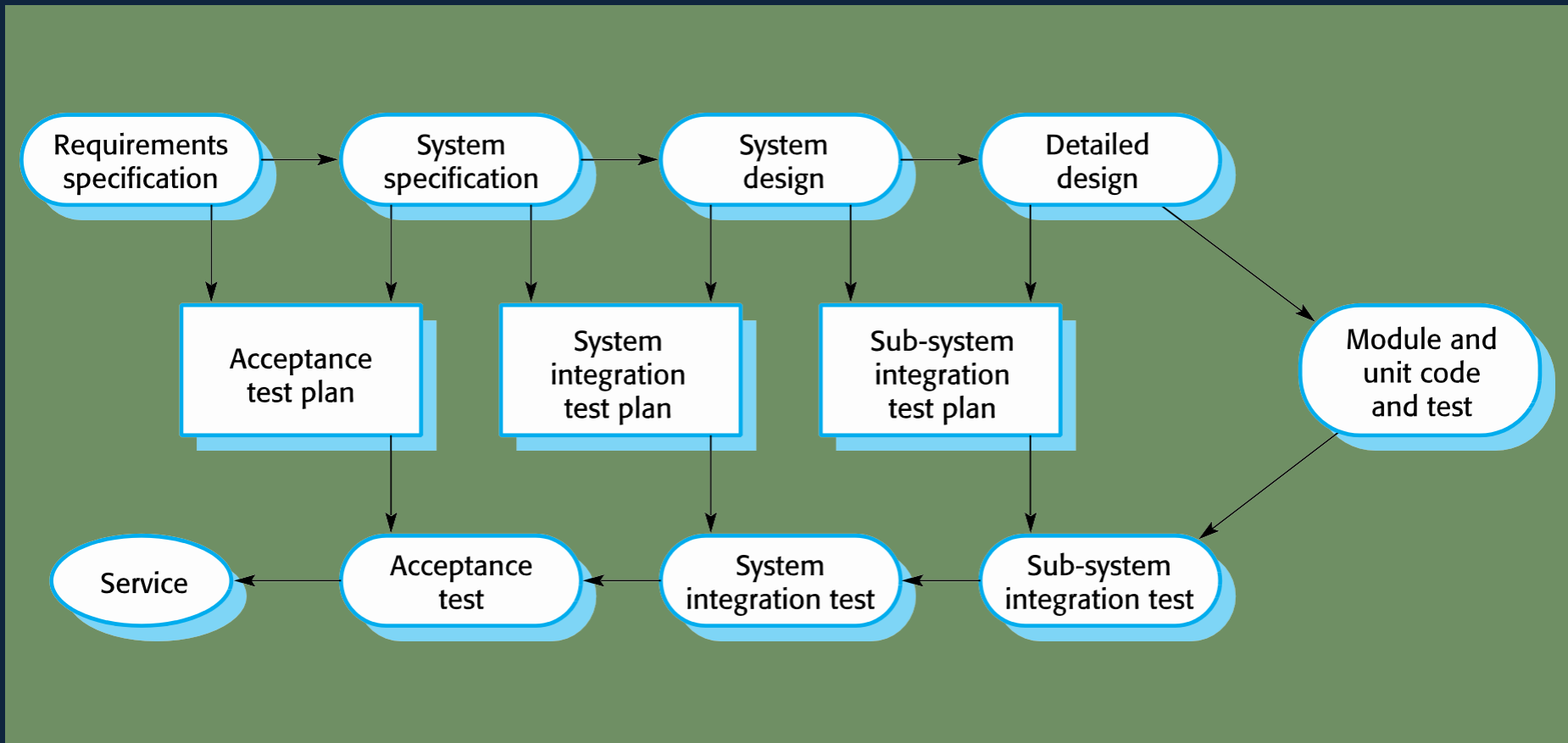
Testing stages (Cont.)

- Normally, component testing is simply part of the normal development process.
- Programmers make up their own test data and incrementally test the code as it is developed.
- The programmer knows the component and is therefore the best person to generate the cases.

Testing stages (Cont.)

- If an *incremental approach* to development is used, each increment should be tested as it is developed, with these tests based on the requirements for that increment.
- In *test-driven* development, which is a normal part of the agile processes, tests are developed along with the requirements before development starts.
 - This helps the tester and developers to understand the requirements and ensures that there are no delays as test cases are created.
- When a *plan-driven* software process is used (e.g., for critical system development), testing is driven by a set of test plans.
 - An independent team of testers works from these test plans, which have been developed from the system specifications and design.

Testing phases in a plan-driven software process



Testing stages

(V-model and Beta testing)

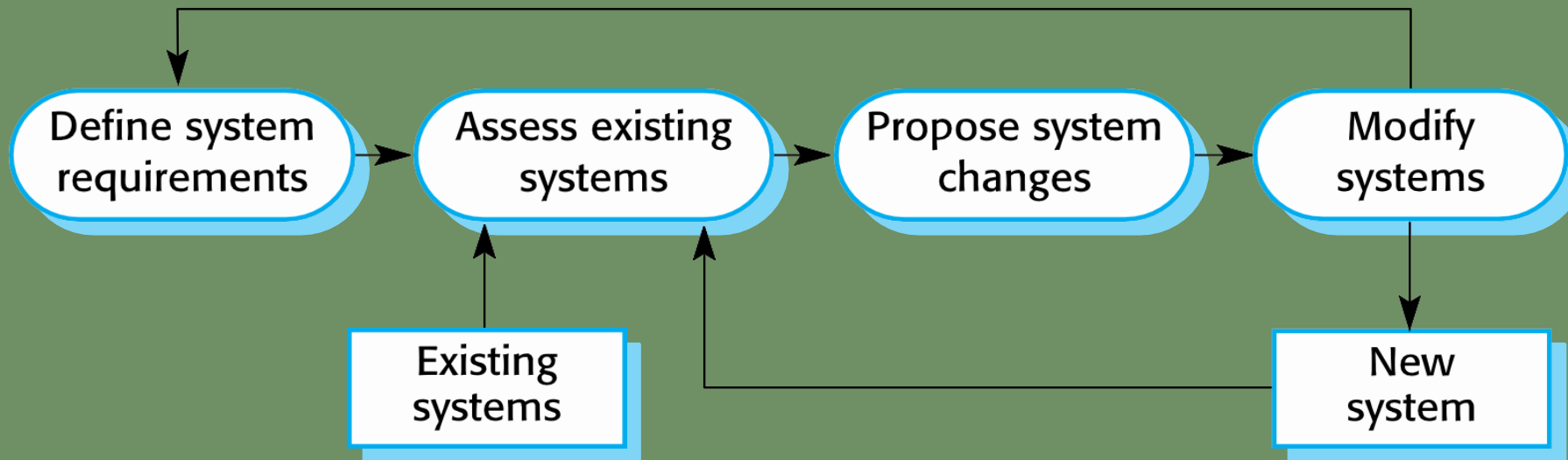
- The process shown in last slide is sometimes called the *V-model* of development (turn it on its side to see V).
- The *V-model* shows the software validation activities that correspond to each stage of the waterfall process model.
- When a system is to be marketed as a software product, a testing process called a *beta testing* is often used.
- Beta testing involves delivering a system to a number of potential customers who agree to use that system.
- They report problems to the system developers.
- This exposes the product to real use and detects errors that may not have been anticipated by the product developers.
- After this feedback, the software product may be modified and released for further beta testing or general sale.



Software evolution

- Software is inherently flexible and can change.
- As requirements change through changing business circumstances, the software that supports the business must also evolve and change.
- Although there has been a demarcation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new.

System evolution





Coping with change

- Change is inevitable in all large software projects.
 - Business changes lead to new and changed system requirements
 - New technologies open up new possibilities for improving implementations
 - Changing platforms require application changes
- Change leads to rework so the costs of change include both rework (e.g. re-analyzing requirements) as well as the costs of implementing new functionality

Reducing the costs of rework

- There are two related approaches that may be used to reduce the cost of rework:
- **Change anticipation**, where the software process includes activities that can anticipate possible changes before significant rework is required.
 - For example, a prototype system may be developed to show some key features of the system to customers.
- **Change tolerance**, where the process is designed so that changes can be accommodated at relatively low cost.
 - This normally involves some form of incremental development. Proposed changes may be implemented in increments that have not yet been developed. If this is impossible, then only a single increment (a small part of the system) may have be altered to incorporate the change.

Ways to Handle coping with changing system requirements

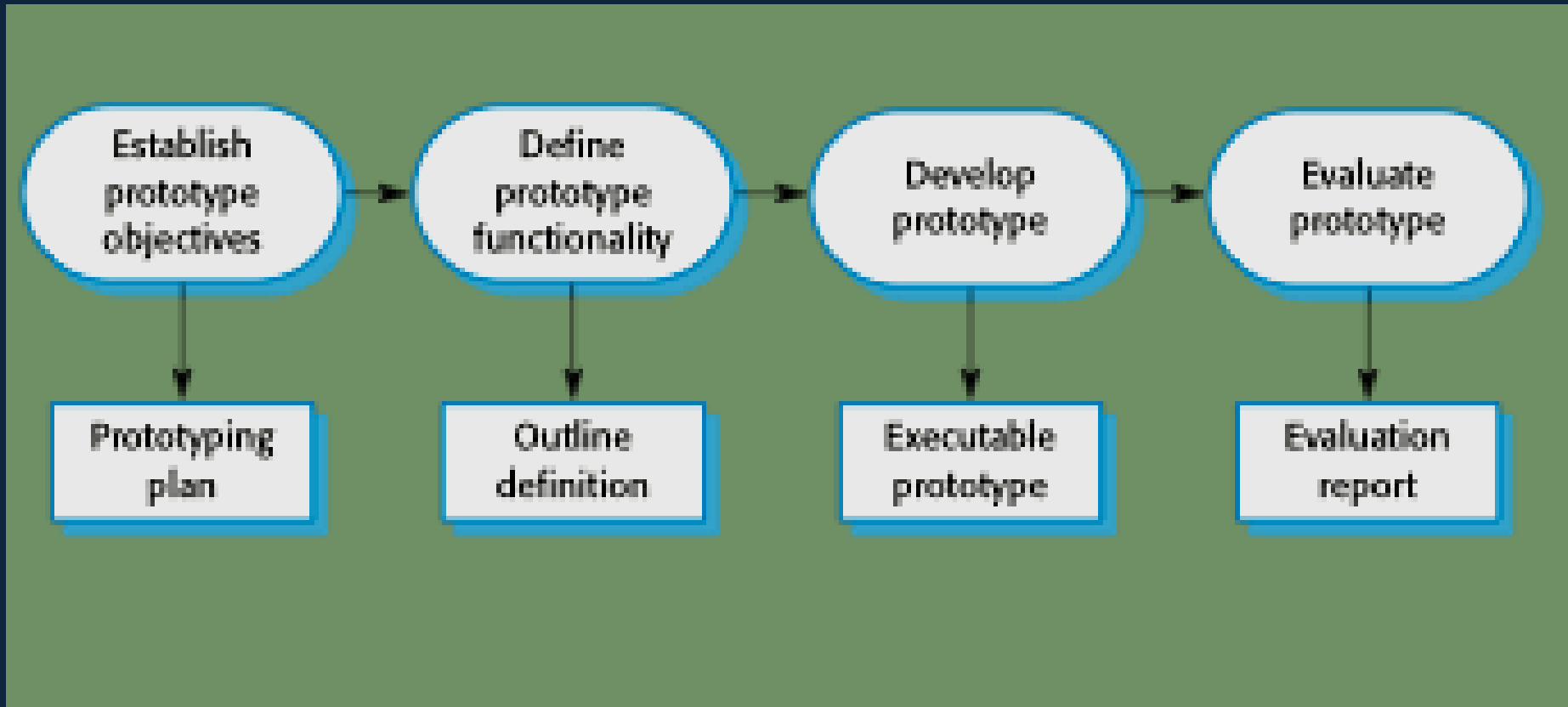
- Here we will discuss two ways of coping with change and changing system requirements:
- System prototyping, where a version of the system or part of the system is developed quickly to check the customer's requirements and the feasibility of some design decisions. This support change avoidance as it allows users to experiment with the system before the delivery and so refine their requirements. The number of requirements change proposals made after delivery is therefore likely to be reduced.
- Incremental delivery, where system increments are delivered to the customer for comment and experimentation. This supports both change avoidance and change tolerance. It avoids the premature commitment to requirements for the whole system and allows changes to be incorporated into later increments at relatively low cost.



Software prototyping

- A prototype is an initial version of a system used to demonstrate concepts and try out design options.
- A prototype can be used in:
 - The requirements engineering process to help with requirements elicitation and validation;
 - In design processes to explore options and develop a User Interface design;
 - In the testing process to run back-to-back tests.

Prototype development process





Benefits of prototyping

- Improved system usability.
- A closer match to users' real needs.
- Improved design quality.
- Improved maintainability.
- Reduced development effort.



Benefits of system prototyping

- System prototypes allow potential users to see how well the system supports their work.
- A system prototype may be used while the system is being designed to carry out design experiments to check the feasibility of a proposed design.



Incremental delivery

- Incremental delivery (Figure 2.10) is an approach to software development where some of the developed increments are delivered to the customer and deployed for use in their working environment.
- In an incremental delivery process, customers define which of the services are most important and which are least important to them.
- A number of delivery increments are then defined, with each increment providing a subset of the system functionality.
- The allocation of services to increments depends on the service priority, with the highest priority services implemented and delivered first.

Incremental delivery

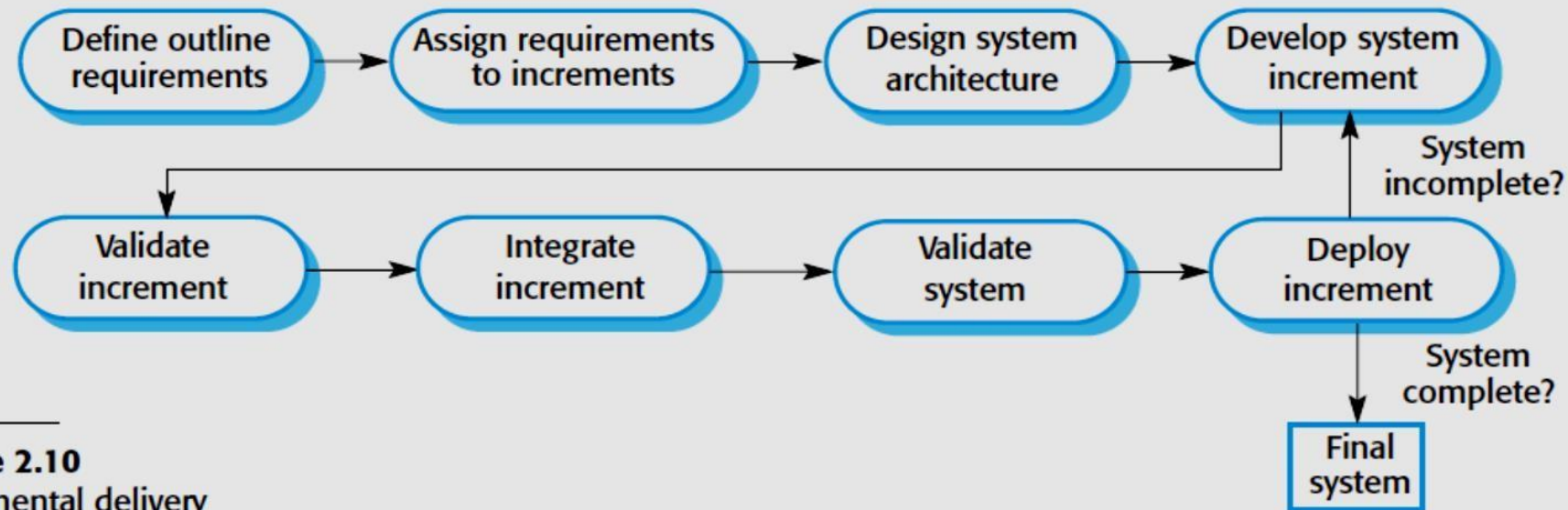


Figure 2.10
Incremental delivery

Advantages of Incremental delivery

- Incremental delivery has a number of advantages:
 - Customers can use the early increments as prototypes and gain experience that informs their requirements for later system increments.

Unlike prototypes, these are part of the real system, so there is no relearning when the complete system is available.
 - Customers do not have to wait until the entire system is delivered before they can gain value from it.

The first increment satisfies their most critical requirements, so they can use the software immediately.

Advantages of Incremental delivery (Cont.)

- Incremental delivery has a number of advantages:
 - The process maintains the benefits of incremental development in that it should be relatively easy to incorporate changes into the system.
 - As the highest priority services are delivered first and later increments then integrated, the most important system services receive the most testing.

This means that customers are less likely to encounter software failures in the most important parts of the system.



Advantages of Incremental delivery

- Incremental delivery has a number of advantages:
 - The process maintains the benefits of incremental development in that it should be relatively easy to incorporate changes into the system.

Incremental delivery problems

- Iterative delivery is problematic when the new system is intended to replace an existing system.
 - Users need all of the functionality of the old system and are usually unwilling to experiment with an incomplete new system.
 - It is often impractical to use the old and new systems alongside each other as they are likely to have different databases and user interfaces.

Incremental delivery problems (Cont.)

- Most systems require a set of basic facilities that are used by different parts of the system.
 - As requirements are not defined in detail until an increment is to be implemented, it can be hard to identify common facilities that are needed by all increments.
- The essence of the iterative process is that the specification is developed in conjunction with the software.
 - However, this conflicts with the procurement model of many organizations, where the complete system specification is part of the system development contract.



Process improvement

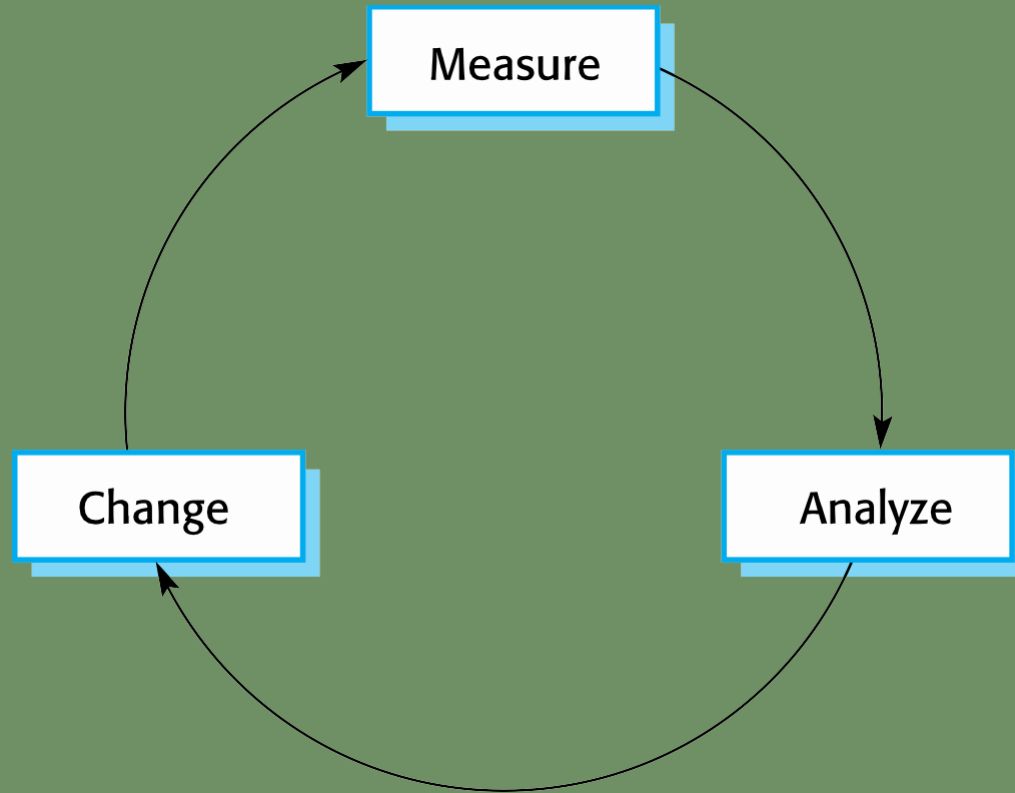
- Many software companies have turned to software process improvement as a way of enhancing the quality of their software, reducing costs, or accelerating their development process.
- Process improvement means understanding existing processes and changing these processes to increase product quality and/or reduce costs and development time.



Approaches to improvement

- The process *maturity approach*, which focuses on improving process and project management and introducing good software engineering practice.
 - The level of process maturity reflects the extent to which good technical and management practice has been adopted in organizational software development processes.
- The *agile approach*, which focuses on iterative development and the reduction of overheads in the software process.
 - The primary characteristics of agile methods are rapid delivery of functionality and responsiveness to changing customer requirements.

The process improvement cycle





Process improvement activities

- *Process measurement*

- You measure one or more attributes of the software process or product. These measurements form a baseline that helps you decide if process improvements have been effective.

- *Process analysis*

- The current process is assessed, and process weaknesses and bottlenecks are identified. Process models (sometimes called process maps) that describe the process may be developed.

- *Process change*

- Process changes are proposed to address some of the identified process weaknesses. These are introduced and the cycle resumes to collect data about the effectiveness of the changes.



Process measurement

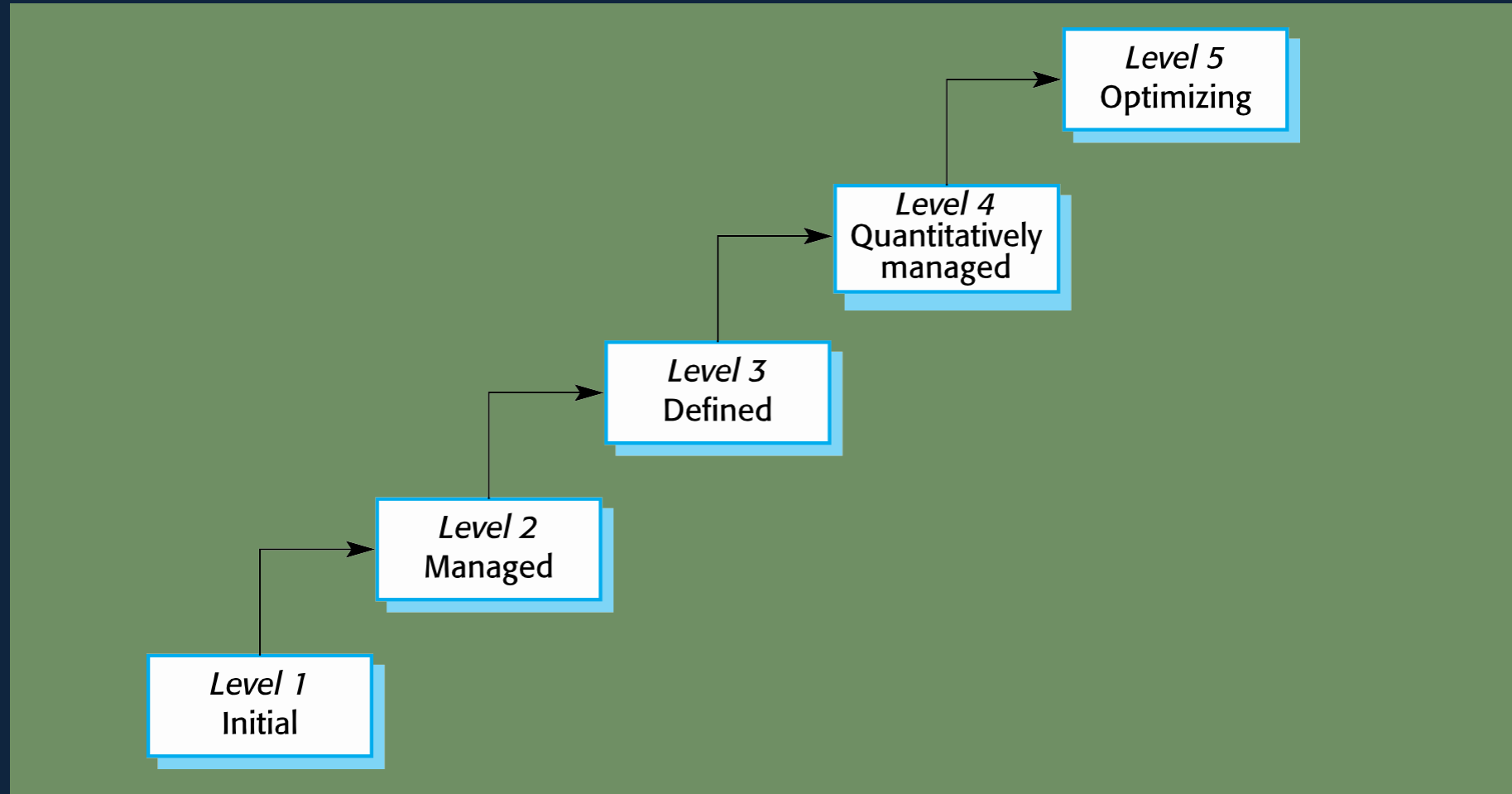
- Wherever possible, quantitative process data should be collected
 - However, where organisations do not have clearly defined process standards this is very difficult as you don't know what to measure. A process may have to be defined before any measurement is possible.
- Process measurements should be used to assess process improvements
 - But this does not mean that measurements should drive the improvements. The improvement driver should be the organizational objectives.



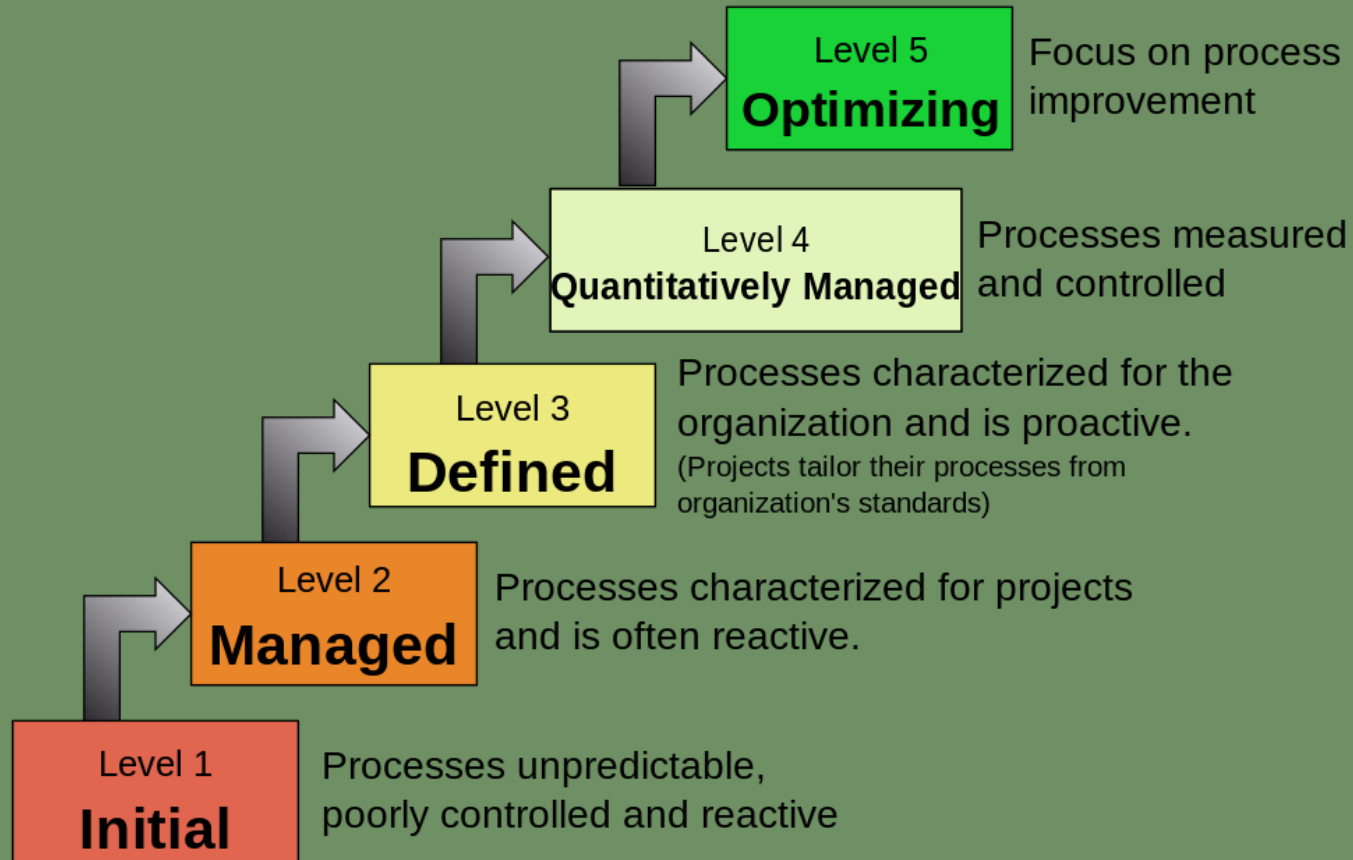
Process metrics

- Time taken for process activities to be completed
 - E.g. Calendar time or effort to complete an activity or process.
- Resources required for processes or activities
 - E.g. Total effort in person-days.
- Number of occurrences of a particular event
 - E.g. Number of defects discovered.

Capability maturity levels



Capability maturity levels



The SEI capability maturity model

■ Initial

- The goal associated with this process area are satisfied, and for all processes the scope of the work to be performed is explicitly set out and communicated to the team members.

■ Managed

- The goals associated with the process area are met, and organizational policies are in place that define when each process should be used.
 - There must be documented project plans that define the project goals.
 - Resources management and process monitoring procedures must be in place across the institution.

■ Defined

- This level focuses on organizational standardization and deployment of processes.
 - Each project has a managed process that is adapted to the project requirements from a defined set of organizational processes.
 - Process assets and process measurements must be collected and used for future process improvements.

The SEI capability maturity model

(Cont.)

■ Quantitatively managed

- At this level, there is an organizational responsibility to use statistical and other quantitative methods to control sub processes.
 - That is, collected process and product measurements must be used in process management.

■ Optimising

- At this highest level, the organization must use the process and product measurements to drive process improvement.
 - Trends must be analysed and the processes adapted to changing business needs.



Key points

- Software processes are the activities involved in producing a software system.
- Software process models are abstract representations of these processes.
- General process models describe the organization of software processes.
- Examples of these general models include the ‘waterfall’ model, incremental development, and reuse-oriented development.
- Generic process models include the ‘waterfall’ model, incremental development, and reuse-oriented development.
- Waterfall model is the first published model of the software development process, also called the Classic Life Cycle or Linear Sequential Model.



Key points

- Incremental development is based on the idea of developing an initial implementation, exposing this to user comment and refining it through many versions until an adequate system has been developed.
- Reuse-oriented approach relies on a large base of reusable software components and some integrating framework for these components.



Key points

- Requirements engineering is the process of developing a software specification.
- Software Specification is the process of understanding and defining what services are required from the system and identifying the constraints on the system's operation and development.
- Design and implementation processes are concerned with transforming a requirements specification into an executable software system.



Key points

- Design and implementation processes are concerned with transforming a requirements specification into an executable software system.
- Software validation is the process of checking that the system conforms to its specification and that it meets the real needs of the users of the system.



Key points

- Software evolution takes place when you change existing software systems to meet new requirements.
- The software must evolve to remain useful.
- Processes should include activities such as prototyping and incremental delivery to cope with change.
- The principal approaches to process improvement are agile approaches, geared to reducing process overheads, and maturity-based approaches based on better process management and the use of good software engineering practice.

A decorative pattern of hexagons in various shades of blue, orange, and white, arranged in a honeycomb-like structure on the left side of the slide.

Thank you

Prof. Dr. Faheem Akhtar Rajput